The Pulickal Framework

Operational Refinement, Behavioral Distinguishability, and Resource Lower Bounds

A task-semantic theory of exact realization, finite capacity, approximation, and architecture-relative impossibility

Pearl Bipin Pulickal

Independent Researcher
Electronics Engineering Graduate, National Institute of Technology Goa

Central question

How sharply can minimum realization resources be characterized from the future-task behavior that a specification requires an implementation to preserve?

September 2026

Contents

1 Introduction **2**

1.1 Reading conventions . . . 3

1.2 The conceptual route . . . 3

1.3 Convergence and the avoidance of infinite regress . . . 3

1.4 Main contributions . . . 4

2 Scope, Claims, and Boundaries **4**

3 Formal Setup: Information States, Tasks, and Signatures **5**

3.1 Information states . . . 5

3.2 Behavioral signatures . . . 6

3.3 A coarsest-sufficient-representation theorem . . . 6

4 Task Enrichment and Specification Refinement **8**

4.1 Enlarging the task family . . . 8

4.2 Operational refinement . . . 8

4.3 Effective refinement . . . 9

5 Exact Realization **10**

5.1 State-based implementations . . . 10

5.2 State separation . . . 10

5.3 Quotient realization and exact minimum . . . 11

5.4 Relationship to automata minimization . . . 12

6 Binary Memory and Information-Style Accounting **12**

6.1 Finite-bit realization . . . 12

6.2 Refinement accumulation . . . 12

6.3 A distinguishability index . . . 13

6.4 Why cardinality is not entropy . . . 13

7 Architecture-Aware Capacity and Resource Lower Bounds **14**

7.1 Implementation classes . . . 14

7.2 Capacity impossibility . . . 14

7.3 Resource optima and certified lower-bound laws . . . 15

7.4 Multiple resources and Pareto structure . . . 16

8 A Hierarchy of Resource Statements **16**

9 The Resource-Bearing Refinement Principle **16**

10 Worked Exact Examples **17**

10.1 Example I: exact binary history retention 17

10.2 Example II: event summaries versus full histories 18

10.3 Example III: numerical precision 18

10.4 Example IV: controller quantization and certification 19

10.5 Example V: abstraction in verification 19

11 Approximate Realization: Packing, Covering, and Error **20**

11.1 Behavioral metric 20

11.2 Packing obstruction 20

11.3 Covering numbers and constructive sufficiency 21

11.4 Approximation frontier 22

11.5 Zero-error limit and exact/approximate unification 22

11.6 Monotonicity 23

11.7 Numerical interval calculation 23

12 Counterexamples to Naive Cost Monotonicity **24**

12.1 Redundant refinement 24

12.2 Compression and structure 24

12.3 Recomputation 24

12.4 External information 24

12.5 Architecture substitution 25

12.6 Task restriction 25

13 Realizability Regions and Design Trade-offs **25**

14 Connections to Established Theory **26**

14.1 Automata and behavioral minimization 26

14.2 Refinement calculus 27

14.3 Abstract interpretation and abstraction 27

14.4 Numerical analysis 27

14.5 Metric entropy and covering theory 27

14.6 Resource-aware synthesis and communication 28

14.7 Physical and quantum computation 28
14.8 Logical incompleteness 28

15 A General Application Procedure **28**

16 Extensions and Open Problems **29**
16.1 Nondeterministic specifications 29
16.2 Probabilistic behavior 29
16.3 Communication-constrained realization 30
16.4 Architecture-specific capacity laws 30
16.5 Approximation rates 30
16.6 Empirical validation 30

17 Limitations and Interpretive Guardrails **31**
17.1 Deterministic main semantics 31
17.2 State completeness 31
17.3 Configuration count is not total engineering cost 31
17.4 Capacity is not sufficiency 31
17.5 Task dependence 31
17.6 No universal scalar for rigor 31
17.7 No universal physical theorem 31

18 Why the Framework Converges **32**
18.1 Semantic closure 32
18.2 Resource closure 32
18.3 The endpoint 32

19 Discussion: What the Framework Actually Establishes **33**
19.1 Level I: semantic statement 33
19.2 Level II: exact state statement 33
19.3 Level III: architecture-capacity statement 33
19.4 Level IV: resource-cost statement 33
19.5 Level V: engineering-optimum statement 33

20 Conclusion **34**

A Notation Summary **35**

B Binary Threshold Table **36**

C Proof Audit and Logical Dependency Map **36**

C.1 Exact theory 36

C.2 Capacity theory 36

C.3 Approximation theory 37

D Design Checklist **37**

E Closing Statement **37**

Abstract

A mathematical specification can become more precise, more expressive, more detailed, or more strongly constrained without thereby becoming more expensive to realize. Conversely, a seemingly small strengthening of a requirement can create a large number of distinctions that an implementation must preserve. The useful object for a resource lower bound is therefore not mathematical detail as such, but the collection of future-task behaviors that remain operationally distinguishable.

This paper develops a task-semantic framework for making that observation precise. A deterministic specification S is modeled by a behavior map $F_S : \mathcal{X} \times \mathcal{T} \to \mathcal{Y}$, where $\mathcal{X}$ is a set of information states and $\mathcal{T}$ is the family of admissible future tasks or continuations. Each state induces a behavioral signature $\sigma_S(x) = F_S(x, \cdot)$. States with identical signatures are operationally equivalent. The resulting quotient $\mathcal{X}/\sim_S$ is the canonical semantic state space for the chosen task family, and its cardinality

$$D(S) := \mathrm{card}(\mathcal{X}/\sim_S)$$

is the number of task-relevant behavioral classes.

The exact theory has two complementary theorems. First, every exact state-based realization must separate distinct operational classes. Second, in an unrestricted deterministic state model, the quotient itself is an exact realization. Hence $D(S)$ is not merely a convenient counting device: it is the exact minimum number of abstract states needed by the specified behavioral semantics. Binary memory then supplies the familiar corollary $b \geq \lceil \log_2 D(S) \rceil$, while architecture-specific capacity functions translate semantic demand into conditional impossibility results. The paper carefully separates this semantic necessity from practical sufficiency and from claims about total engineering cost.

The framework further treats refinement as partition refinement. A refinement cannot decrease $D(S)$, but an increase in D alone still does not imply that every concrete implementation becomes more expensive. Strict growth of a chosen resource lower bound requires an independently justified architecture-specific lower-bound law. This distinction permits compression, recomputation, external information, and architectural substitution to be treated as legitimate counterexamples to naive monotone-cost claims.

For approximate realization, exact cardinality is replaced by metric separation. Behavioral packing numbers yield impossibility certificates: if more than K signatures are pairwise separated by more than 2ε, no K-configuration realization can have worst-case error at most ε. Covering numbers provide the complementary constructive side in the unrestricted response model. The result is an explicit approximation frontier rather than a binary exact-feasible/impossible slogan.

The framework is deliberately convergent. Its semantic quotient closes the question of which states are operationally equivalent for the stated deterministic task family. Further layers are introduced only when a new resource, architecture, probabilistic semantics, or approximation criterion is explicitly placed under study. The result is not a universal law about mathematical rigor, physical information, or engineering cost, but a semantics-to-resource methodology whose conclusions can be tested theorem by theorem and instantiated architecture by architecture.

Keywords. operational semantics; behavioral equivalence; refinement; state minimization; resource lower bounds; finite-state realization; configuration capacity; metric packing; covering numbers; approximation; formal verification; numerical precision; engineering feasibility.

1 Introduction

Mathematical specifications and engineering implementations inhabit different descriptive regimes. A specification may use exact real numbers, arbitrary tolerances, idealized continuity, quantified properties, symbolic invariants, or very large state spaces. An implementation must operate through a selected physical and computational architecture: finite memory, bounded arithmetic, limited bandwidth, finite sensing resolution, finite execution time, finite energy, reliability constraints, or some combination thereof.

A familiar slogan attempts to summarize this tension:

> *"The more rigorous the mathematics becomes, the less useful it becomes in engineering."*

The slogan is attractive because it gestures toward a real phenomenon, but it is much too strong to support a theorem. Mathematical refinement can expose symmetry, identify invariants, improve conditioning, clarify interfaces, simplify algorithms, support formal verification, and reveal cheaper implementations. Refinement calculus, abstract interpretation, finite- precision analysis, and model reduction all contain examples in which greater mathematical structure improves engineering practice [1, 2, 3, 4].

The sharper question is therefore:

> *When does a strengthened specification create task-relevant behavioral distinctions whose exact preservation imposes an unavoidable resource requirement on a specified implementation class?*

The word *task-relevant* is decisive. A document can become longer without changing anything an implementation is ever asked to distinguish. Conversely, one additional sentence can require a system to answer a future question in two different ways where previously one answer sufficed. What matters to the lower-bound problem is the induced partition of possible information states by future behavior.

The framework developed here consequently separates four interfaces:

$$\begin{aligned} &\text{specification} \longrightarrow \text{future-task semantics} \\ \longrightarrow\ &\text{semantic state requirement} \longrightarrow \text{architecture-specific resource bound.} \end{aligned}$$

Each arrow has a different logical status. The first two define semantics. The third is an exact theorem about state-based realization. The fourth is a conditional engineering theorem whose strength depends on the implementation class and its independently justified capacity or cost model.

This separation is more than a stylistic choice. It prevents several common category errors. A semantic distinction is not automatically a memory bit. A larger quotient is not automatically a larger circuit. A larger lower bound for one architecture is not a physical law. And the existence of an exact semantic quotient does not imply that an implementation can be built cheaply, quickly, reliably, or even at all within a practical architecture.

1.1 Reading conventions

The notation in this paper is intentionally layered. Symbols such as $D(S)$ describe semantic demand; symbols such as $\mathrm{Cap}_{\mathcal{A}}(\mathbf{B})$ describe architectural supply; symbols such as $C^{\star}_{\mathcal{A}}(S)$ describe an optimization problem over implementations. Keeping these roles separate prevents a lower bound from silently turning into an exact cost model. [1]

The word "state" is used in two related but distinct senses. An *information state* $x \in \mathcal{X}$ belongs to the problem being specified. An *implementation configuration* $z \in \mathcal{Z}_I$ belongs to a realization. The central question is precisely how many distinct implementation configurations are required to preserve the distinctions present in the information states.

1.2 The conceptual route

The exact theory can be read as a sequence of questions:

1. What information states may arise?
2. What future tasks may subsequently be presented?
3. Which states have exactly the same required answer to every such task?
4. How many equivalence classes remain after those operationally irrelevant distinctions are collapsed?
5. What configuration capacity can the chosen implementation architecture supply?
6. What resource lower bound follows from that capacity?

The order matters. A lower bound introduced before the task semantics is fixed can measure syntactic description size rather than operational necessity.

1.3 Convergence and the avoidance of infinite regress

A natural philosophical objection is that a lower-bound argument could itself become recursively demanding: one defines a semantic distinction, then asks for a deeper measure of semantic complexity, then a deeper measure of that measure, and so on.

The present framework deliberately does not take that route. For the deterministic task model studied here, the behavioral quotient closes the semantic question at the required level. Two states are equivalent exactly when they have identical answers to all admissible future tasks. The quotient therefore represents the coarsest state partition that is sufficient for those tasks. Once this object is fixed, resource analysis asks a new question: what can a specified architecture encode or recover?

This is not a claim that no deeper semantics exists in philosophy or computer science. It is a statement of methodological closure: *no additional notion is needed in order to establish the exact state-separation theorem for the model currently under study.* A deeper theory becomes relevant only when one enlarges the problem, for example to probabilistic behavior, communication complexity, latency, energy, robustness, or a different notion of approximation.

[1] This separation is methodological rather than merely terminological. A mathematically exact statement can still be too abstract to predict a specific implementation cost. Conversely, an empirical engineering model can be highly useful without itself being an exact semantic theorem.

1.4 Main contributions

The paper develops the following connected results.

1. **Task-semantic signatures.** A specification induces a future-behavior signature on every information state, making operational equivalence explicit and task-relative.
2. **Canonical quotient state space.** The quotient by identical signatures is the coarsest semantically sufficient state representation. Its cardinality is the exact minimum state count in the unrestricted deterministic model.
3. **Refinement theory.** Operational refinement is treated as partition refinement, yielding monotonicity of the semantic state count and a natural quotient map between refined and coarser semantic states.
4. **Architecture-relative lower bounds.** A configuration-capacity function converts semantic state demand into conditional impossibility. A separate lower-bound function is used when one wants statements about memory, time, energy, area, precision, or other resources.
5. **Approximate realization.** Packing numbers yield lower bounds on the number of required configurations, while covering numbers give constructive upper bounds in the unrestricted response model. Exact realization appears as the zero-error endpoint.
6. **Failure modes of naive monotonicity.** Redundant refinement, compression, recomputation, external information, architecture substitution, and changes in the task family are treated as explicit counterexamples to the claim that more mathematics must mean more engineering cost.

The familiar binary-bit formula is thus deliberately demoted from being the conceptual core to being a special case:

$$b \geq \lceil \log_2 D(S) \rceil .$$

The substantive question is not why the logarithm exists; that follows from binary capacity. The substantive question is why a particular value of $D(S)$ is the correct semantic demand for the application at hand.

2 Scope, Claims, and Boundaries

A rigorous lower-bound framework is defined as much by what it does *not* claim as by what it proves.

The framework does *not* claim that:

1. mathematical rigor is intrinsically opposed to engineering usefulness;
2. a longer or more detailed mathematical specification necessarily costs more to implement;
3. every refinement increases the optimal cost of every implementation;
4. every semantic distinction must be stored as a literal memory bit;
5. a state lower bound for one architecture is a universal physical law;

6. mathematical description length is the same object as operational information;
7. Shannon entropy, Kolmogorov complexity, metric entropy, and operational distinguishability are interchangeable;
8. Gödel incompleteness or quantum uncertainty is required for the core lower-bound argument.

The exact impossibility pattern instead has the conditional form

$$D(S) > \mathrm{Cap}_{\mathcal{A}}(\mathbf{B}) \quad \Longrightarrow \quad S \text{ has no exact realization in } \mathcal{A} \text{ under budget } \mathbf{B}. \tag{1}$$

The semantic quantity $D(S)$ is also not a property of an isolated formula. It is a property of a specification *together with* its admissible future tasks. Changing the task family can change the quotient without changing the underlying physical phenomenon.

The word "unavoidable" always has a scope. In this paper it means unavoidable *within the stated realization model*. If an external database, communication channel, recomputation procedure, side information, or alternative physical mechanism is allowed but omitted from the model, a purported lower bound may simply be a theorem about the wrong system.

3 Formal Setup: Information States, Tasks, and Signatures

3.1 Information states

Let $\mathcal{X}$ be a nonempty set of information states. The element $x \in \mathcal{X}$ may represent an input prefix, a retained measurement history, a controller mode, a partial state estimate, a logical state of a proof procedure, or any other situation from which future tasks may be posed.

No assumption is made that $\mathcal{X}$ is finite. Indeed, many motivating applications involve continuous or combinatorially enormous state spaces.

Let $\mathcal{T}$ be a nonempty set of admissible future tasks, continuations, or queries. The purpose of $\mathcal{T}$ is to specify precisely what the system may later be asked to do.

Let $\mathcal{Y}$ be the outcome space. In the main exact theory, the outcome for every state and task is deterministic.

Definition 3.1 (Task semantics)**.** A deterministic specification S is a map

$$F_S : \mathcal{X} \times \mathcal{T} \to \mathcal{Y}. \tag{2}$$

For $x \in \mathcal{X}$ and $t \in \mathcal{T}$, the value $F_S(x, t)$ is the exact outcome required by the specification when state x is subsequently subjected to task t.

The abstraction is intentionally modest. It says nothing yet about hardware,

algorithms, memory representation, timing, energy, or physical storage. Those ingredients enter only after semantic demand has been characterized.

3.2 Behavioral signatures

Definition 3.2 (Behavioral signature)**.** For $x \in \mathcal{X}$, define the behavioral signature of x under S by

$$\sigma_S(x) := F_S(x, \cdot) \in \mathcal{Y}^{\mathcal{T}}. \tag{3}$$

The signature records the complete future behavior required over the admissible task family.

Thus the map

$$\sigma_S : \mathcal{X} \to \mathcal{Y}^{\mathcal{T}}$$

compresses each information state into the function that describes everything the model considers operationally relevant about that state.

Definition 3.3 (Operational equivalence)**.** For $x, y \in \mathcal{X}$, define

$$x \sim_S y \iff F_S(x,t) = F_S(y,t) \quad \text{for every } t \in \mathcal{T}. \tag{4}$$

Equivalently,

$$x \sim_S y \iff \sigma_S(x) = \sigma_S(y).$$

Proposition 3.4 (Equivalence)**.** *The relation $\sim_S$ is an equivalence relation on $\mathcal{X}$.*

Proof. Reflexivity and symmetry follow from equality. For transitivity, if $x \sim_S y$ and $y \sim_S z$, then for every $t \in \mathcal{T}$,

$$F_S(x,t) = F_S(y,t) = F_S(z,t),$$

hence $x \sim_S z$. □

Definition 3.5 (Behavioral signature space)**.** Let

$$\Sigma_S := \operatorname{range}(\sigma_S) \subseteq \mathcal{Y}^{\mathcal{T}}. \tag{5}$$

When the quotient is finite, define the operational distinguishability count by

$$D(S) := \operatorname{card}(\mathcal{X}/\sim_S) = \operatorname{card}(\Sigma_S). \tag{6}$$

The equality of the two cardinalities follows because σ_S maps each equivalence class to exactly one signature and different classes have different signatures.

Remark 3.6. $D(S)$ does not count equations, symbols, variables, pages, proof steps, lemmas, or lines of prose. It counts the number of distinct future behaviors that remain after states with identical task behavior have been identified. This is why it is better interpreted as a semantic state count than as a generic measure of "information".

3.3 A coarsest-sufficient-representation theorem

The quotient can be characterized without mentioning implementation architecture at all.

Definition 3.7 (Semantically sufficient representation)**.** Let $h : \mathcal{X} \to \mathcal{H}$ be any map into a set $\mathcal{H}$. We say that h is semantically sufficient for $(S, \mathcal{T})$ if

$$h(x) = h(y) \implies \sigma_S(x) = \sigma_S(y) \qquad \forall x, y \in \mathcal{X}. \tag{7}$$

In words, equal representation values may not collapse states that the task semantics can distinguish.

Theorem 3.8 (Canonical sufficiency and minimality)**.** *Define*

$$q_S : \mathcal{X} \to \mathcal{X}/\!\sim_S, \qquad q_S(x) = [x]_{\sim_S}. \tag{8}$$

Then q_S is semantically sufficient. Moreover, for every semantically sufficient representation $h : \mathcal{X} \to \mathcal{H}$, there exists a unique map

$$\gamma : \mathrm{range}(h) \to \Sigma_S$$

such that

$$\gamma(h(x)) = \sigma_S(x) \qquad \forall x \in \mathcal{X}. \tag{9}$$

Consequently,

$$\mathrm{card}(\mathrm{range}(h)) \geq D(S) \tag{10}$$

whenever $D(S)$ is finite.

Proof. If $q_S(x) = q_S(y)$, then $x \sim_S y$, so $\sigma_S(x) = \sigma_S(y)$. Thus q_S is semantically sufficient.

Now let h be semantically sufficient. Define

$$\gamma(h(x)) := \sigma_S(x).$$

To show that γ is well-defined, suppose $h(x) = h(y)$. By semantic sufficiency, $\sigma_S(x) = \sigma_S(y)$, so the proposed value of γ is independent of the chosen representative. The factorization (9) follows by construction. Uniqueness follows because every element of $\mathrm{range}(h)$ has the form $h(x)$ for some x.

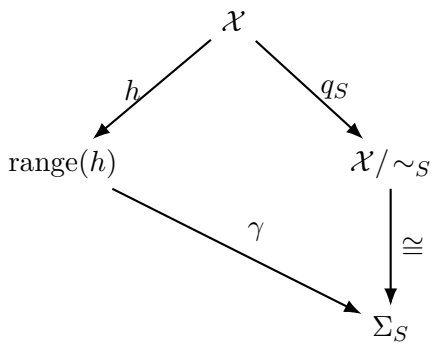
Since γ is a surjection from $\mathrm{range}(h)$ onto Σ_S, one has

$$\mathrm{card}(\mathrm{range}(h)) \geq \mathrm{card}(\Sigma_S) = D(S).$$

□

This theorem is useful because it identifies the quotient as more than a counting trick. The quotient is the *coarsest semantically sufficient representation*: any other exact semantic representation must have at least as many representation values.

One can summarize the theorem diagrammatically:



The diagram makes the convergence point explicit: the quotient is the semantic object that remains after all operationally irrelevant distinctions have been removed.

4 Task Enrichment and Specification Refinement

4.1 Enlarging the task family

Suppose $\mathcal{T}_1 \subseteq \mathcal{T}_2$ and the same underlying behavior map is used on the enlarged task set. We then obtain two equivalence relations, $\sim_{S,\mathcal{T}_1}$ and $\sim_{S,\mathcal{T}_2}$.

Proposition 4.1 (Task-monotonicity)**.** *If $\mathcal{T}_1 \subseteq \mathcal{T}_2$, then*

$$x \sim_{S,\mathcal{T}_2} y \quad \Longrightarrow \quad x \sim_{S,\mathcal{T}_1} y. \tag{11}$$

Consequently, whenever both quotients are finite,

$$D(S, \mathcal{T}_2) \geq D(S, \mathcal{T}_1). \tag{12}$$

Proof. Equality on the larger task family implies equality on its subset. Therefore the larger task family induces a partition that is at least as fine. A finer finite partition has at least as many classes. □

This is a foundational point for applications. The same physical system can have radically different semantic state demands under different admissible future queries.

Example 4.2 (A diagnostic recorder)**.** Suppose a recorder retains the history of a binary sensor. If the only future task is "Was there ever an alarm?", many histories collapse to two classes. If the future task is instead "Report the entire history exactly," every binary history of length n is distinct. The physical sensor need not change; the task family does.

4.2 Operational refinement

Classical refinement relations vary across formal methods. The present theory needs only a semantic condition.

Definition 4.3 (Operational refinement)**.** Let S and S' share $(\mathcal{X}, \mathcal{T}, \mathcal{Y})$. We say that S' is an operational refinement of S, written

$$S' \succeq S, \tag{13}$$

when

$$x \sim_{S'} y \quad \Longrightarrow \quad x \sim_S y \qquad \forall x, y \in \mathcal{X}. \tag{14}$$

Thus the refined specification may distinguish states that the coarse specification identifies, but it may not merge a pair that the coarse specification already requires to distinguish.

Proposition 4.4 (Refinement monotonicity)**.** *If $S' \succeq S$ and both distinguishability counts are finite, then*

$$D(S') \geq D(S). \tag{15}$$

Proof. Every S'-equivalence class is contained in one S-equivalence class. Define

$$r : \mathcal{X}/\sim_{S'} \to \mathcal{X}/\sim_S, \qquad r([x]_{S'}) = [x]_S.$$

The refinement condition guarantees that r is well-defined, and every coarse class contains an S'-class, so r is surjective. Therefore the domain has cardinality at least that of the codomain. □

Proposition 4.5 (Refinement composition)**.** *If $S'' \succeq S'$ and $S' \succeq S$, then $S'' \succeq S$.*

Proof. The implication of equivalences composes:

$$x \sim_{S''} y \implies x \sim_{S'} y \implies x \sim_S y.$$

□

The refinement relation is therefore a preorder at the level of semantic partitions. It need not be antisymmetric as a relation on syntactically different specifications: two specifications may induce the same partition while requiring different names, formulas, or presentation conventions.

Definition 4.6 (Operational equivalence of specifications)**.** Write

$$S' \approx S$$

when $\sim_{S'} = \sim_S$. This relation identifies specifications that induce the same operational partition, even if their syntactic descriptions differ.

This observation prevents an unnecessary confusion between *semantic change* and *document change*.

4.3 Effective refinement

Definition 4.7 (Operationally effective refinement)**.** An operational refinement $S' \succeq S$ is operationally effective when

$$D(S') > D(S). \tag{16}$$

For finite quotients, strict inequality means that at least one previously merged pair of states has become task-relevant.

It is important not to infer an engineering-cost theorem from this definition. The following implications are generally invalid without further premises:

$$D(S') > D(S) \implies \text{memory increases},$$

$$D(S') > D(S) \implies \text{time increases},$$

$$D(S') > D(S) \implies \text{energy increases},$$

or

$$D(S') > D(S) \implies \text{total implementation cost increases}.$$

The framework proves semantic necessity first and asks for a separate resource law afterward.

5 Exact Realization

5.1 State-based implementations

An implementation is modeled abstractly enough to cover digital state machines, software objects, controllers, embedded systems, lookup structures, and other deterministic state-based realizations.

Definition 5.1 (Implementation model)**.** An implementation I consists of:

1. a configuration set $\mathcal{Z}_I$;
2. a configuration map
$$\phi_I : \mathcal{X} \to \mathcal{Z}_I;$$
3. a response map
$$G_I : \mathcal{Z}_I \times \mathcal{T} \to \mathcal{Y}.$$

The configuration $\phi_I(x)$ represents the persistent information available when future tasks are executed.

The model separates *what is retained* from *how the response is computed*. This is deliberate: a state lower bound should not silently assume a particular algorithmic representation of the response.

Assumption 5.2 (State completeness)**.** Every persistent source of information on which future responses may depend is included in the modeled configuration or explicitly represented as part of the implementation architecture and its allowed channels.

The assumption is a scope condition, not a universal assertion about real systems.[2] An implementation that can contact a server, consult a database, measure a fresh quantity, or recompute information from retained raw data must place that mechanism inside the model if the lower bound is intended to cover it.

Definition 5.3 (Exact realization)**.** An implementation I exactly realizes S if

$$G_I(\phi_I(x), t) = F_S(x, t) \qquad \forall x \in \mathcal{X},\ \forall t \in \mathcal{T}. \tag{17}$$

5.2 State separation

Theorem 5.4 (State-separation theorem)**.** *If I exactly realizes S, then*

$$x \not\sim_S y \quad \Longrightarrow \quad \phi_I(x) \neq \phi_I(y). \tag{18}$$

Consequently, if $D(S)$ is finite,

$$\operatorname{card}(\mathcal{Z}_I) \geq D(S). \tag{19}$$

[2] In particular, the phrase "persistent information" should be read relative to the observation point at which future tasks are posed. A later measurement or query is not persistent state unless the model explicitly treats it as such.

Proof. Suppose $x \not\sim_S y$. Then there exists $t \in \mathcal{T}$ such that

$$F_S(x,t) \neq F_S(y,t).$$

Assume for contradiction that $\phi_I(x) = \phi_I(y)$. Exact realization gives

$$F_S(x,t) = G_I(\phi_I(x),t) = G_I(\phi_I(y),t) = F_S(y,t),$$

contradicting the choice of t. Therefore distinct operational classes must map to distinct configurations. Choosing one configuration for each semantic class gives at least $D(S)$ configurations. □

This proof is elementary, but the exact role of the assumptions matters more than the algebra. The theorem does not say that every irrelevant detail must be absent from an implementation. An implementation can store redundant information. It says only that *distinct task-relevant classes cannot share a configuration.*

5.3 Quotient realization and exact minimum

Theorem 5.5 (Semantic quotient realization)**.** *Assume $D(S) < \infty$. Let*

$$\mathcal{Z}^\star := \mathcal{X}/\!\sim_S, \qquad \phi^\star(x) := [x]_{\sim_S}, \tag{20}$$

and define

$$G^\star([x]_{\sim_S}, t) := F_S(x,t). \tag{21}$$

Then $G^\star$ is well-defined, the implementation $(\mathcal{Z}^\star, \phi^\star, G^\star)$ exactly realizes S, and

$$\operatorname{card}(\mathcal{Z}^\star) = D(S). \tag{22}$$

Hence $D(S)$ is the minimum number of states among deterministic state-based realizations with no architecture-specific restriction on state representation or response computation.

Proof. If $[x]_{\sim_S} = [y]_{\sim_S}$, then $x \sim_S y$, so

$$F_S(x,t) = F_S(y,t) \qquad \forall t \in \mathcal{T}.$$

Thus $G^\star([x],t)$ is independent of the chosen representative and is well-defined. Exactness follows immediately:

$$G^\star(\phi^\star(x),t) = G^\star([x],t) = F_S(x,t).$$

The state count is exactly the cardinality of the quotient, namely $D(S)$. The state-separation theorem shows that no exact deterministic state-based implementation can use fewer configurations. □

Remark 5.6 (The semantic center of the framework)**.** The quotient-realization theorem is the conceptual center of the exact theory. The quantity $D(S)$ is not introduced merely to make a later logarithm look convenient. It is the exact minimum state count of the abstract deterministic behavioral semantics. The engineering problem begins only when one asks how those semantic states can be represented and operated upon by a restricted architecture.

5.4 Relationship to automata minimization

The construction is structurally related to the state-minimization logic of finite automata and the Myhill–Nerode theorem: histories that have identical future acceptance behavior may be merged, whereas distinguishable histories must remain separate [5, 6, 7]. The present framework does not claim to replace automata theory, nor to solve state minimization for arbitrary computational models. Instead, it abstracts the same behavioral principle to a generic task/output semantics and then connects the resulting semantic state requirement to architecture-relative resource models.

A useful way to state the relationship is:

$$\text{Myhill–Nerode-like future equivalence} \quad \rightsquigarrow \quad \text{generic task-semantic quotient.}$$

The contribution claimed here lies in carrying that quotient explicitly into an engineering lower-bound workflow and in distinguishing semantic state necessity from architecture-specific resource cost.

6 Binary Memory and Information-Style Accounting

6.1 Finite-bit realization

If a state consists of at most b binary bits, it has at most 2^b configurations. Combining this observation with theorem 5.4 gives the familiar bound.

Corollary 6.1 (Finite-bit lower bound)**.** *Any exact implementation using at most b binary state bits satisfies*

$$2^b \geq D(S), \tag{23}$$

and hence

$$\boxed{b \geq \lceil \log_2 D(S) \rceil \,.} \tag{24}$$

The logarithm is therefore a property of binary capacity. It is not the definition of semantic distinguishability.

6.2 Refinement accumulation

Consider a sequence

$$S_0 \preceq S_1 \preceq \cdots \preceq S_n \tag{25}$$

with finite counts $D_i := D(S_i)$. Define, whenever $D_{i-1} > 0$,

$$q_i := \frac{D_i}{D_{i-1}}. \tag{26}$$

Proposition 6.2 (Multiplicative and logarithmic accumulation)**.** *For every such finite sequence,*

$$D_n = D_0 \prod_{i=1}^{n} q_i, \tag{27}$$

and therefore

$$\boxed{\log_2 D_n = \log_2 D_0 + \sum_{i=1}^{n} \log_2 q_i.} \tag{28}$$

Proof. The product identity telescopes:

$$D_0 \prod_{i=1}^{n} \frac{D_i}{D_{i-1}} = D_n.$$

Taking logarithms yields equation (28). □

The mathematical identity is simple. Its useful interpretation is that the semantic demand itself can be additive on a logarithmic scale.

For binary refinements with $q_i = 2$,

$$D_n = 2^n D_0, \qquad b_n \geq \lceil \log_2 D_0 \rceil + n. \tag{29}$$

For uniform r-way refinement, $r \geq 2$,

$$D_n = D_0 r^n, \qquad b_n \geq \lceil \log_2 D_0 + n \log_2 r \rceil . \tag{30}$$

Remark 6.3 (No one-bit-per-refinement theorem)**.** Even if every refinement strictly increases D, the integer bit requirement need not increase at every step. For example, the lower bound is two bits for both $D = 3$ and $D = 4$. What accumulates continuously is $\log_2 D$, while the minimum integer number of binary state bits changes at thresholds.

6.3 A distinguishability index

For finite $D(S)$ define

$$\mathrm{Dist}(S) := \log_2 D(S). \tag{31}$$

This quantity is useful as an accounting coordinate because multiplicative state-class growth becomes additive.

It should not be confused with Shannon entropy. There is no probability distribution, no averaging, and no typical-case assumption in equation (31). It is a worst-case cardinality index:

$$\mathrm{Dist}(S) = \log_2(\text{number of task-distinguishable classes}).$$

This distinction becomes essential when moving from finite exact state problems to probabilistic sources, where Shannon information and operational task complexity may answer different questions.

6.4 Why cardinality is not entropy

The quantity $\mathrm{Dist}(S)$ is deliberately distribution-free. Suppose the semantic quotient contains D classes but the operating environment visits one class almost always. A Shannon entropy computed from a highly concentrated source distribution can be much smaller than $\log_2 D$.

Neither quantity is "more correct" in isolation: they answer different questions. The present framework asks for a worst-case guarantee over the admissible information states and tasks.

Similarly, a count of syntactic possibilities can exceed $D(S)$ by an arbitrary factor if many syntactically distinct cases have identical future behavior. The quotient removes those distinctions before any resource calculation begins.

Remark 6.4 (What the framework measures)**.** The semantic state count should therefore be interpreted as a *worst-case representation requirement for future-task behavior*, not as a universal information quantity. This distinction is what permits the same framework to coexist with Shannon information, communication complexity, description-length measures, and domain-specific complexity notions without identifying them.

7 Architecture-Aware Capacity and Resource Lower Bounds

The semantic quotient tells us the minimum number of abstract state classes. It does not yet tell us how many transistors, bytes, gates, clock cycles, joules, square millimeters, or communication symbols are required. That translation belongs to the architecture layer.

7.1 Implementation classes

Definition 7.1 (Implementation class and resource budget)**.** An implementation class $\mathcal{A}$ is a set of admissible implementations. A resource usage vector is

$$\mathbf{R}(I) = (R_{\text{mem}}, R_{\text{time}}, R_{\text{energy}}, R_{\text{area}}, R_{\text{bw}}, R_{\text{prec}}, \ldots). \tag{32}$$

A vector budget $\mathbf{B}$ is satisfied when

$$\mathbf{R}(I) \preceq \mathbf{B}$$

componentwise. Scalar-resource problems are recovered as a special case.

Definition 7.2 (Configuration capacity)**.** For a stated configuration abstraction, define

$$\text{Cap}_{\mathcal{A}}(\mathbf{B}) := \sup\{\text{card}(\mathcal{Z}_I) : I \in \mathcal{A},\ \mathbf{R}(I) \preceq \mathbf{B}\}, \tag{33}$$

whenever the supremum is well-defined in the chosen cardinality model.

For an ordinary B-bit state register,

$$\text{Cap}(B) = 2^B.$$

For an unconstrained abstract machine the capacity could be unbounded. Such an abstraction is mathematically legitimate but not automatically physically informative: finite observation, noise, reliability, write precision, and readout restrictions may dominate the real system.

7.2 Capacity impossibility

Theorem 7.3 (Capacity realizability criterion)**.** *Let $D(S)$ be finite. If*

$$D(S) > \text{Cap}_{\mathcal{A}}(\mathbf{B}), \tag{34}$$

then S has no exact realization in $\mathcal{A}$ satisfying the budget $\mathbf{R}(I) \preceq \mathbf{B}$.

Proof. Every exact realization requires at least $D(S)$ configurations by theorem 5.4. The capacity definition states that no admissible implementation under the budget can provide more than $\mathrm{Cap}_{\mathcal{A}}(\mathbf{B})$ configurations. The strict inequality therefore makes exact realization impossible. □

The converse is deliberately not claimed:

$$D(S) \leq \mathrm{Cap}_{\mathcal{A}}(\mathbf{B}) \tag{35}$$

is necessary but generally not sufficient. The architecture may lack the right response function, may violate latency constraints, may have insufficient numerical precision, or may fail reliability requirements.

7.3 Resource optima and certified lower-bound laws

For a scalar resource measure R define

$$C^{\star}_{\mathcal{A}}(S) := \inf\{R(I) : I \in \mathcal{A},\ I \text{ exactly realizes } S\}, \tag{36}$$

with $C^{\star}_{\mathcal{A}}(S) = +\infty$ if no exact implementation exists.

Definition 7.4 (Certified lower-bound function)**.** Let $\mathcal{S}$ be a class of specifications. A function

$$g_{\mathcal{A}} : \mathbb{N} \to [0, \infty]$$

is a certified distinguishability-based lower bound on $\mathcal{S}$ if

$$C^{\star}_{\mathcal{A}}(S) \geq g_{\mathcal{A}}(D(S)) \qquad \forall S \in \mathcal{S}. \tag{37}$$

This definition makes the logical burden explicit. The framework itself does not conjure $g_{\mathcal{A}}$ from D. The architecture or an independent theorem must supply it.

Definition 7.5 (Certifiably resource-bearing refinement)**.** A refinement $S' \succeq S$ is certifiably resource-bearing with respect to $(\mathcal{A}, R)$ if there exists a valid lower-bound function $g_{\mathcal{A}}$ such that

$$g_{\mathcal{A}}(D(S')) > g_{\mathcal{A}}(D(S)). \tag{38}$$

Proposition 7.6. *If S' is certifiably resource-bearing, then the certified lower bound on the selected resource strictly increases. This does not imply that every implementation becomes more expensive, nor that the optimum for another architecture or another resource measure increases.*

Proof. Apply equation (37) to S' and S and use equation (38). □

This distinction is the precise mathematical version of the paper's central caution:

semantic growth $\neq$ automatic engineering-cost growth.

7.4 Multiple resources and Pareto structure

A realistic design problem often has several competing resources:

$$\mathbf{R}(I) = (R_{\text{mem}}, R_{\text{time}}, R_{\text{energy}}, R_{\text{area}}, R_{\text{bw}}, R_{\text{prec}}, \dots).$$

There may be no single scalar that faithfully represents all trade-offs. Reducing memory can increase computation; increasing precision can reduce algorithmic robustness requirements; improving latency can increase area or energy.

The natural object is therefore often a Pareto set. The framework recommends reporting componentwise lower bounds or stating an explicit scalarization rather than silently treating all resources as one number.

8 A Hierarchy of Resource Statements

The framework is easiest to misuse when several logically different statements are compressed into one sentence. The following hierarchy is therefore useful.

Level	Valid statement
Semantic	$S' \succeq S$ and $D(S') > D(S)$ means the refined specification retains more task-relevant behavioral distinctions.
State	Any exact state-based realization must provide at least $D(S)$ configurations.
Capacity	If the architecture supplies fewer than $D(S)$ admissible configurations, exact realization is impossible.
Resource	A strict lower-bound increase follows only when an independently justified function $g_{\mathcal{A}}$ increases with D.
Engineering optimum	The true optimum may be larger because of constraints not represented by the state count or capacity function.

The framework therefore does not identify

$$D(S), \quad \log_2 D(S), \quad \mathrm{Cap}_{\mathcal{A}}(\mathbf{B}), \quad C^{\star}_{\mathcal{A}}(S)$$

as interchangeable quantities. They belong to different layers of the argument.

A useful test for any proposed theorem is to ask which arrow it proves:

$$\boxed{\text{semantics} \to \text{state necessity} \to \text{capacity obstruction} \to \text{resource cost.}}$$

If an argument silently jumps over an arrow, it requires an additional premise.

9 The Resource-Bearing Refinement Principle

The preceding theory can be condensed without overstating it.

A mathematical refinement is not intrinsically resource-costly. It becomes certifiably resource-bearing when it creates task-relevant behavioral distinctions and a specified implementation class admits a proven resource lower bound that increases with those distinctions.

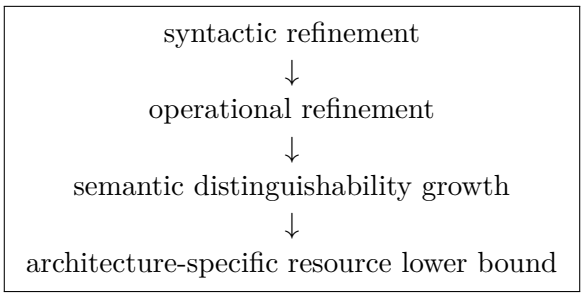
The word *certifiably* is essential. The principle contains two distinct mathematical ingredients:

$$S' \succeq S, \quad D(S') > D(S) \quad \Longrightarrow \quad \text{new semantic distinctions must be preserved,} \tag{39}$$

$$g_{\mathcal{A}}(D(S')) > g_{\mathcal{A}}(D(S)) \quad \Longrightarrow \quad \text{the certified lower bound rises.} \tag{40}$$

The second implication cannot be obtained merely by repeating the first. It requires an architecture-specific resource theorem.

This yields a useful four-layer separation:



A mathematical document can change at the first layer while leaving all later layers unchanged. A carefully designed specification can jump directly to a large $D(S)$ without becoming syntactically elaborate.

10 Worked Exact Examples

10.1 Example I: exact binary history retention

Consider a recorder observing a binary sequence of length n and then facing only the task

$$\text{report the complete history exactly.}$$

Let

$$\mathcal{X}_n = \{0,1\}^n, \qquad \mathcal{T} = \{\text{report}\}, \qquad F_n(x, \text{report}) = x.$$

If $x \neq y$, the required report differs. Hence every state is its own operational class:

$$D_n = 2^n.$$

Therefore

$$b_n \geq n.$$

With an eight-bit state budget, $n = 8$ is compatible with the counting lower bound, while $n = 9$

is not:

$$D_9 = 512 > 256 = 2^8.$$

The impossibility is exact within the stated eight-bit state model.

The conclusion has nothing to do with mathematical sophistication. It comes from one additional independently recoverable binary distinction.

10.2 Example II: event summaries versus full histories

The previous example becomes more revealing if the task family is changed.

Suppose the same recorder stores a binary history $x = (x_1, \ldots, x_n)$, but the only task is

$$t_\exists : \quad \text{“Was any alarm observed?”}$$

with

$$F(x, t_\exists) = \begin{cases} 1, & \exists i : x_i = 1, \\ 0, & \text{otherwise.} \end{cases}$$

Then

$$D = 2,$$

because all histories containing at least one 1 are operationally equivalent for this single task, as are all-zero histories.

Thus the same physical input family admits:

$$D_{\text{summary}} = 2, \qquad D_{\text{full}} = 2^n.$$

This is a concrete demonstration that semantic state demand belongs to the pair

$$(\text{system}, \text{future-task family}),$$

not to the system description alone.

10.3 Example III: numerical precision

Consider a scalar quantity

$$x \in [a, a + L]$$

and a future report task requiring absolute error at most ε.

The exact behavioral signature can be taken to encode the ideal value itself, while approximate realizations may replace exact signatures by reports within distance ε. If two values differ by more than 2ε, no single scalar output can be within ε of both.

A grid therefore gives a separated family with cardinality on the order of L/ε, and consequently a binary state lower bound with logarithmic precision scaling:

$$B = \Omega\left(\log_2 \frac{L}{\varepsilon}\right).$$

The order notation is important. It does not say that a sensor must contain a literal number of

bits equal to the mathematical lower bound in every physical realization. Coding, estimation, noise, external information, and architecture may alter the concrete design.

If the tolerance is reduced by a factor of 100, the associated logarithmic scale changes by

$$\log_2(100) \approx 6.64.$$

That number is best interpreted as a semantic distinguishability increment in the stated model, not as a universal prescription to add seven physical storage bits to every sensor.

10.4 Example IV: controller quantization and certification

Consider

$$x_{k+1} = Ax_k + Bu_k, \qquad u_k = Kx_k,$$

and an implementation

$$\widehat{K} = K + \Delta K.$$

Finite-word-length effects can influence stability margins and performance [16]. The framework does not conclude from this fact alone that a more precise mathematical representation requires more memory.

Instead, define a task family whose outputs are certified guarantees, for example whether the closed loop satisfies a specified stability margin. Two parameter states are operationally distinct precisely when they imply different required guarantees for some admissible certification task.

The semantic quotient then answers the first question:

which controller parameter states must remain distinct?

An architecture theorem must answer the second:

how much memory, precision, computation, or numerical robustness
is needed to preserve those distinctions?

The distinction turns the informal statement "the new mathematical theorem is too precise for the controller" into the sharper engineering proposition

required certification distinctions $>$ available architecture-specific capacity.

10.5 Example V: abstraction in verification

Suppose a high-fidelity verification model has state space $\mathcal{X}_H$ and an abstraction map

$$\alpha : \mathcal{X}_H \to \mathcal{X}_L.$$

The correct question is not whether $\mathcal{X}_H$ is large in a syntactic sense. It is whether states merged by α have different future behavior with respect to the property and continuations actually used by the verification procedure.

If

$$\alpha(x) = \alpha(y)$$

but x and y are operationally equivalent for all relevant tasks, the merging is semantically harmless in the present model. If a counterexample reveals a distinguishing continuation, the abstraction must refine the partition.

This creates a conceptual bridge to counterexample-guided abstraction refinement: abstraction begins with a coarser partition, and evidence of a property-relevant distinction restores a previously collapsed semantic class [11, 12].

The hardware state space and the verifier's abstract state space remain different objects. A proof-state explosion and a hardware memory lower bound should not be conflated.

11 Approximate Realization: Packing, Covering, and Error

Exact equality is often too strict for engineering. A system may be permitted to produce outputs within a tolerance, preserve a stability margin only up to a specified slack, or approximate a continuous response.

The correct replacement for exact cardinality is a metric on future behavior.

11.1 Behavioral metric

Assume that $\mathcal{Y}$ carries a metric d_Y. The signature space $\mathcal{Y}^{\mathcal{T}}$ then admits the extended sup-metric

$$\rho(\sigma, \tau) := \sup_{t \in \mathcal{T}} d_Y(\sigma(t), \tau(t)), \tag{41}$$

whenever this supremum is finite for the signatures under consideration.

The metric is task-sensitive because the signatures themselves are generated by the task family.

Definition 11.1 (Approximate realization)**.** An implementation I is an ε-realization of S if

$$\rho\big(\widehat{\sigma}_I(x), \sigma_S(x)\big) \le \varepsilon \qquad \forall x \in \mathcal{X}, \tag{42}$$

where

$$\widehat{\sigma}_I(x) := G_I(\phi_I(x), \cdot).$$

Definition 11.2 (Behavioral packing number)**.** For $\alpha > 0$, define the α-packing number

$$M_S(\alpha) := \sup\left\{m : \exists x_1, \dots, x_m \in \mathcal{X} \text{ with } \rho(\sigma_S(x_i), \sigma_S(x_j)) > \alpha \ \forall i \neq j\right\}. \tag{43}$$

The use of strict separation is convenient for the triangle-inequality argument below.

11.2 Packing obstruction

Theorem 11.3 (Approximate state-separation theorem)**.** *If an implementation has at most K configurations and*

$$M_S(2\varepsilon) > K, \tag{44}$$

then no such implementation is an ε-realization of S.

Proof. Choose more than K states with pairwise signature distance greater than 2ε. By the

pigeonhole principle, two of these states, say x_i and x_j, must share a configuration:

$$\phi_I(x_i) = \phi_I(x_j).$$

Hence

$$\widehat{\sigma}_I(x_i) = \widehat{\sigma}_I(x_j) =: \widehat{\sigma}.$$

If I were an ε-realization, the triangle inequality would give

$$\rho(\sigma_S(x_i), \sigma_S(x_j)) \le \rho(\sigma_S(x_i), \widehat{\sigma}) + \rho(\widehat{\sigma}, \sigma_S(x_j)) \le 2\varepsilon,$$

contradicting the selected separation. □

Corollary 11.4 (Approximate bit condition)**.** *An ε-accurate implementation using B state bits must satisfy*

$$M_S(2\varepsilon) \le 2^B. \tag{45}$$

The exact theorem is recovered at zero tolerance in the discrete setting: large exact distinguishability requires many configurations.

11.3 Covering numbers and constructive sufficiency

Packing gives a lower bound. A complementary upper-side object is a covering number.

Definition 11.5 (Behavioral covering number)**.** Let $\Sigma_S \subseteq \mathcal{Y}^{\mathcal{T}}$ be the signature set. Define

$$N_S(\varepsilon) := \inf \left\{ m : \exists c_1, \dots, c_m \in \mathcal{Y}^{\mathcal{T}} \text{ such that } \Sigma_S \subseteq \bigcup_{j=1}^{m} \mathbb{B}_\rho(c_j, \varepsilon) \right\}. \tag{46}$$

The centers c_j are not required to correspond to signatures of actual states. This matters: an optimal approximate response may lie between ideal signatures.

Theorem 11.6 (Unrestricted covering realization)**.** *Suppose the implementation model permits an arbitrary deterministic response map $G : \{1, \dots, K\} \times \mathcal{T} \to \mathcal{Y}$. If*

$$N_S(\varepsilon) \le K,$$

then there exists a K-configuration ε-realization of S.

Proof. Choose centers $c_1, \dots, c_m$ with $m \le K$ that cover Σ_S. For each $x \in \mathcal{X}$, choose an index $j(x)$ such that

$$\rho(\sigma_S(x), c_{j(x)}) \le \varepsilon.$$

Define

$$\phi(x) := j(x)$$

and set

$$G(j, t) := c_j(t).$$

Then

$$\widehat{\sigma}_I(x) = c_{j(x)},$$

so by construction

$$\rho(\widehat{\sigma}_I(x), \sigma_S(x)) \leq \varepsilon$$

for every x. □

Combining the two theorems yields a clean sandwich:

$$M_S(2\varepsilon) \leq N_S(\varepsilon) \quad \text{(as a lower-bound relation)}, \tag{47}$$

and, in the unrestricted deterministic model,

$$N_S(\varepsilon) \leq K$$

is sufficient for an ε-realization.

The two quantities need not coincide. The gap reflects the geometry of the behavioral signature set and the fact that efficient covering centers need not be themselves ideal signatures.

11.4 Approximation frontier

Define the unrestricted K-configuration approximation threshold by

$$\varepsilon^{\star}_{\text{unres}}(S, K) := \inf\{\varepsilon \geq 0 : N_S(\varepsilon) \leq K\}. \tag{48}$$

The packing obstruction gives the certified lower threshold

$$\underline{\varepsilon}_{\text{pack}}(S, K) := \inf\{\varepsilon \geq 0 : M_S(2\varepsilon) \leq K\}. \tag{49}$$

Then

$$\varepsilon^{\star}_{\text{unres}}(S, K) \geq \underline{\varepsilon}_{\text{pack}}(S, K), \tag{50}$$

because no error smaller than the packing threshold can be achieved.

This formulation is preferable to claiming that the packing expression is the exact engineering error curve. It is a certified obstruction. The covering quantity supplies a constructive upper characterization in the unrestricted response model. Additional architecture restrictions can make the true engineering optimum larger.

11.5 Zero-error limit and exact/approximate unification

The covering construction also clarifies the relationship between the exact and approximate theories.

Proposition 11.7 (Zero-error covering identity)**.** *If $D(S) < \infty$, then*

$$N_S(0) = D(S). \tag{51}$$

Proof. A radius-zero ball contains exactly one behavioral signature. Hence at least $D(S)$ centers are needed to cover Σ_S. Conversely, choosing one center equal to each signature covers the set with exactly $D(S)$ centers. □

Thus the exact theory is the zero-error endpoint of the covering theory:

$$\boxed{D(S) = N_S(0).}$$

This is more than a pleasing analogy. It gives a single geometric object whose zero-radius limit recovers exact state minimization and whose positive-radius behavior quantifies approximation.

Lemma 11.8 (Packing–covering obstruction)**.** *For every $\varepsilon \geq 0$ for which the quantities are defined,*

$$M_S(2\varepsilon) \leq N_S(\varepsilon). \tag{52}$$

Proof. Suppose an ε-cover uses m centers. Any two signatures lying in the same radius-ε ball are at distance at most 2ε by the triangle inequality. Therefore a family whose pairwise distances are strictly greater than 2ε can contain at most one element per covering ball. Hence every 2ε-packing has cardinality at most m. Taking the infimum over covers yields the result. □

The standard packing lower bound is consequently the geometric shadow of a covering-based constructive characterization.

11.6 Monotonicity

The packing number satisfies

$$\alpha_1 \leq \alpha_2 \quad \Longrightarrow \quad M_S(\alpha_2) \leq M_S(\alpha_1).$$

Similarly,

$$\varepsilon_1 \leq \varepsilon_2 \quad \Longrightarrow \quad N_S(\varepsilon_1) \geq N_S(\varepsilon_2)$$

whenever the covering numbers are finite.

Thus tighter accuracy demands finer behavioral resolution. This is the approximate analogue of the exact statement that finer semantic partitions require at least as many state classes.

11.7 Numerical interval calculation

For a scalar interval of length L under absolute-error tolerance ε, consider ideal signatures corresponding to the scalar values themselves. A set of points separated by more than 2ε yields a packing. Therefore

$$M_S(2\varepsilon) \gtrsim \frac{L}{2\varepsilon},$$

up to endpoint conventions, and hence

$$B \gtrsim \log_2 \frac{L}{\varepsilon}.$$

This scaling is the operational origin of logarithmic precision requirements. The engineering representation need not literally be a uniform grid; the lower bound applies to any state-based representation whose future behavior must distinguish the selected packing.

12 Counterexamples to Naive Cost Monotonicity

A good lower-bound framework should prove its own limits by exhibiting cases where tempting converses fail.[3]

12.1 Redundant refinement

Suppose S' adds explanatory lemmas, identities, annotations, or proof obligations that do not change the future outcomes of any admissible task. Then

$$\sim_{S'} = \sim_S \qquad \text{and} \qquad D(S') = D(S).$$

Mathematical quality can improve without semantic state demand changing.

12.2 Compression and structure

A refinement can reveal algebraic structure, symmetry, conservation laws, or correlations that allow a more efficient representation. Then semantic distinguishability may increase while concrete implementation cost remains constant or decreases.

The state-separation theorem does not forbid compression. It forbids only the collapse of task-distinguishable states into the same persistent configuration within the specified realization model.

12.3 Recomputation

A system can store less and compute more. Thus

$$R_{\text{mem}} \downarrow$$

does not imply

$$R_{\text{time}} \downarrow \quad \text{or} \quad R_{\text{energy}} \downarrow .$$

A lower memory bound is not a theorem about total cost.

12.4 External information

Suppose a future task is allowed to consult an external source containing information omitted from the local state. The local device may then have a smaller configuration space while exact behavior remains achievable.

The right response is not to declare the lower bound false. The right response is to include the external channel in the architecture model and ask whether the relevant resource is now communication, storage elsewhere, latency, or some other quantity.

[3] This is not merely rhetorical caution. Counterexamples delimit the exact hypotheses under which a theorem may legitimately be reused in another engineering domain.

12.5 Architecture substitution

A binary memory lower bound does not imply impossibility for an analog, hybrid, distributed, stochastic, or otherwise different architecture unless that architecture is also ruled out by an independently established capacity bound.

This is why architecture-relative language is not a weakness. It is what prevents an abstract state theorem from being mistaken for an unsupported physical universal.

12.6 Task restriction

An exact implementation may become dramatically cheaper if the future task family is restricted. This is not an evasion of the framework. It is a direct semantic operation:

$$\mathcal{T}_{\text{small}} \subseteq \mathcal{T}_{\text{large}} \quad \Longrightarrow \quad D_{\text{small}} \leq D_{\text{large}}.$$

The design response "change the task" is therefore mathematically distinct from "buy more memory".

13 Realizability Regions and Design Trade-offs

Let R be a scalar resource and $\mathcal{A}$ an implementation class.

Definition 13.1 (Exact feasible set)**.** For budget B define

$$\mathfrak{F}_0(\mathcal{A}, B) := \{S : \exists I \in \mathcal{A},\ R(I) \leq B,\ I \text{ exactly realizes } S\}. \tag{53}$$

Definition 13.2 (Approximate feasible set)**.** For $\varepsilon \geq 0$ define

$$\mathfrak{F}_\varepsilon(\mathcal{A}, B) := \{S : \exists I \in \mathcal{A},\ R(I) \leq B,\ I \text{ is an } \varepsilon\text{-realization of } S\}. \tag{54}$$

Proposition 13.3 (Tolerance monotonicity)**.** *If*

$$0 \leq \varepsilon_1 \leq \varepsilon_2,$$

then

$$\mathfrak{F}_{\varepsilon_1}(\mathcal{A}, B) \subseteq \mathfrak{F}_{\varepsilon_2}(\mathcal{A}, B).$$

Proof. Every ε_1-realization is automatically an ε_2-realization because a smaller error is also at most the larger tolerance. □

The framework therefore exposes a multidimensional design frontier:

increase resources
or
relax tolerance
or
change architecture
or
reduce task-relevant distinctions
or
restrict the task family.

These operations are mathematically different.

Design operation	What changes in the framework?
More resources	The admissible implementation set expands through a larger budget.
Higher tolerance	The metric approximation requirement becomes weaker.
Architecture change	The configuration capacity and response constraints change.
Abstraction	The semantic partition deliberately becomes coarser.
Task restriction	The set of future distinctions to be answered becomes smaller.

This is the framework's engineering interpretation of the familiar "feasible / infeasible" boundary: an infeasible exact point can often be made feasible by moving in a different coordinate rather than simply increasing memory.

14 Connections to Established Theory

The framework is a synthesis rather than an attempt to claim ownership of several classical ideas.

14.1 Automata and behavioral minimization

The strongest structural precedent is the Myhill–Nerode theorem and related sequential-machine theory. Equivalent histories can be merged when all future acceptance behavior agrees, and the resulting finite quotient corresponds to the state count of the minimal deterministic automaton [5, 6, 7].

The present abstraction replaces language acceptance with a general outcome map

$$F_S : \mathcal{X} \times \mathcal{T} \to \mathcal{Y}$$

and then connects the quotient to architecture-relative resource models.

A modern literature also continues to develop Myhill–Nerode-like behavioral characterizations for richer automata, including symbolic and register settings [8]. This reinforces the point that

behavioral equivalence is a well-established structural idea; the present contribution should therefore be understood as a disciplined extension of that idea into a semantics-to-resource methodology.

14.2 Refinement calculus

Refinement calculus studies systematic development from more abstract to more concrete specifications [1]. The present framework is compatible with that tradition but uses a narrower semantic relation: what matters for the resource theory is whether the refinement changes the future-behavior partition.

This allows the framework to remain agnostic about the particular syntax or proof calculus used to describe the specification.

14.3 Abstract interpretation and abstraction

Abstract interpretation provides a general mathematical theory of sound abstraction and approximation [2]. The present quotient is not an abstract interpretation lattice, nor does it attempt to replace that theory.

The connection is instead conceptual: an abstraction is useful when it merges states that are irrelevant to a specified semantic property. When a counterexample reveals a property-relevant difference, refinement restores a distinction. The task-semantic quotient provides a compact language in which to state that principle.

14.4 Numerical analysis

Finite-precision numerical analysis studies representation error, stability, conditioning, and algorithmic sensitivity [3]. The Pulickal Framework does not provide a replacement for those analyses. It asks a question one level earlier: which numerical distinctions are actually required by the future tasks?

This is especially relevant in engineering problems where a mathematical specification may state a stronger precision property than the downstream task can observe.

14.5 Metric entropy and covering theory

Packing and covering numbers are classical tools for quantifying the complexity of sets in metric spaces and function spaces. Kolmogorov and Tikhomirov's foundational work formalized ε-entropy and ε-capacity for function classes [9].

The present framework specializes the same geometric logic to the signature set

$$\Sigma_S \subseteq \mathcal{Y}^{\mathcal{T}}.$$

The novelty claim is therefore not the invention of packing or covering numbers. It is the explicit identification of the relevant metric object as a space of task-induced behaviors and the connection of that object to state-based realization and architecture capacity.

14.6 Resource-aware synthesis and communication

Resource-aware program synthesis directly incorporates resource restrictions into implementation search [10]. Communication complexity likewise derives lower bounds from the amount of task-relevant information that must be conveyed.

These traditions strengthen the framework's methodological message: *the resource model must be specified before a lower bound can be meaningfully interpreted.*

14.7 Physical and quantum computation

The distinction between abstract computation and physical realization is studied in the philosophy and theory of computation [13]. Quantum resource theories organize tasks, allowed operations, and resource monotones operationally [14].

These connections are motivational rather than foundational. No universal thermodynamic, quantum, or philosophical premise is used in the proofs of the exact state-separation or approximate packing theorems.

14.8 Logical incompleteness

Gödel's incompleteness theorems concern formal provability and consistency [15]. The present theory concerns realizability under an explicit state and resource model. Any analogy between the two is structural at most. Incompleteness is not a premise of the lower-bound theory.

15 A General Application Procedure

The framework can be applied to an engineering problem in a fixed sequence.

1. **State the information-state space.** Specify exactly what counts as a possible retained or latent situation before the future task.
2. **State the future-task family.** List the actual questions, continuations, queries, certification requests, or downstream computations the implementation must support.
3. **Define the behavior map.** Give $F_S(x, t)$ explicitly, or provide a sound task-level abstraction.
4. **Form the behavioral quotient.** Determine or bound
$$D(S) = |\mathcal{X}/\sim_S|.$$
5. **State all allowed information channels.** Include external databases, recomputation, sensors, communication, shared memory, and side information whenever they are admissible.
6. **Choose the implementation class.** Specify the allowed representation, timing, reliability, precision, and other architectural constraints.

7. **Derive capacity or resource bounds.** Prove or import a function such as

$$\mathrm{Cap}_{\mathcal{A}}(\mathbf{B})$$

or a lower-bound function

$$g_{\mathcal{A}}(D).$$

8. **Compare semantic demand with capacity.** Use the exact or approximate obstruction theorems.

9. **If exact realization fails, define the metric.** Study packing and covering of behavioral signatures.

10. **Choose the design response.** Add resources, change architecture, restrict tasks, abstract distinctions, or relax tolerances.

The order prevents circularity. One does not need a mysterious additional "rigor measure" between the quotient and the architecture theorem. The quotient answers the semantic distinction question. A resource theorem answers the implementation question. The chain is finite because each stage has a different subject matter.

16 Extensions and Open Problems

The deterministic exact theory is intentionally small. Several meaningful extensions remain open.

16.1 Nondeterministic specifications

If a specification assigns a set of acceptable outputs rather than one exact output to each (x, t)), equality of sets is not automatically the right merge criterion. Two states may admit overlapping acceptable behaviors but still require different implementations under a refinement interpretation.

A general extension therefore requires a task-level notion of safe merging, possibly based on:

$$\text{set inclusion}, \quad \text{refinement}, \quad \text{simulation}, \quad \text{or another preorder.}$$

The deterministic theory here is the clean base case.

16.2 Probabilistic behavior

For stochastic systems, signatures could become probability laws over future outcomes:

$$\sigma_S(x)(t) \in \mathcal{P}(\mathcal{Y}).$$

A statistical distance—for example total variation, Wasserstein distance, or a task-specific decision loss—could replace the deterministic metric.

The important methodological point is that the choice should come from the application. There is no universal reason to privilege one stochastic metric.

16.3 Communication-constrained realization

Instead of requiring all information to remain local, one could study implementations that transmit enough task-relevant information to a future processor. The semantic quotient remains the same, while the architecture changes from storage capacity to communication capacity.

This suggests a broad resource family:

$$\text{storage}, \quad \text{communication}, \quad \text{computation}, \quad \text{energy}, \quad \text{latency}.$$

The same semantic demand may feed different lower-bound theorems depending on the architecture.

16.4 Architecture-specific capacity laws

The abstract function $\mathrm{Cap}_{\mathcal{A}}$ is deliberately easy to state and often difficult to instantiate sharply. Future work should derive explicit capacity laws for:

- finite-precision sensor interfaces;
- noisy or approximate memories;
- embedded processors;
- distributed storage systems;
- communication-limited controllers;
- hardware accelerators;
- mixed-signal architectures.

The hard part is not counting abstract states. It is proving that a particular architecture cannot encode or recover those states through another mechanism.

16.5 Approximation rates

For a specification family S_λ, one may study

$$\log M_{S_\lambda}(\alpha) \qquad \text{and} \qquad \log N_{S_\lambda}(\alpha)$$

as $\alpha \downarrow 0$ or as λ changes.

The growth rate may be polynomial, exponential, logarithmic, or governed by a more complicated geometric quantity. This is where the framework moves beyond elementary finite counting into genuine asymptotic theory.

16.6 Empirical validation

A useful empirical program would select engineering domains with independently measurable costs, derive a semantic lower bound before implementation, and compare it to Pareto-efficient designs.

Such experiments would not prove the abstract theorems. They would test whether semantic distinguishability is an informative predictor of practical resource bottlenecks.

17 Limitations and Interpretive Guardrails

17.1 Deterministic main semantics

The central exact theory assumes deterministic future behavior. Probabilistic and nondeterministic settings require richer semantic relations.

17.2 State completeness

The state lower bound applies only to the modeled channels. External information, recomputation, shared memory, future sensing, and communication must be represented if they are allowed.

17.3 Configuration count is not total engineering cost

A minimal number of abstract configurations can be small even when the response function is computationally expensive. Conversely, a large configuration space does not imply that every state consumes a proportionally large amount of physical memory.

17.4 Capacity is not sufficiency

The inequality

$$D(S) \leq \mathrm{Cap}_{\mathcal{A}}(\mathbf{B})$$

is generally only necessary. Correct response-function implementation, latency, reliability, precision, energy, and software constraints can still prevent realization.

17.5 Task dependence

$D(S)$ is a property of the specification *and* its task semantics. Changing $\mathcal{T}$ changes the equivalence relation even when the underlying physical system is unchanged.

17.6 No universal scalar for rigor

The framework deliberately refuses to collapse formal detail, generality, proof strength, fidelity, robustness, precision, and abstraction into one scalar notion of "rigor."

17.7 No universal physical theorem

An architecture-relative lower bound becomes a physical engineering claim only after the architecture, budget, noise assumptions, measurement model, and allowed channels have been independently justified.

18 Why the Framework Converges

It is useful to state explicitly why the framework does not need an endless tower of additional semantic measures.

18.1 Semantic closure

The quotient

$$\mathcal{X}/\sim_S$$

answers a sharply delimited question:

> *Which information states are indistinguishable with respect to every future task admitted by the model?*

If two states lie in the same quotient class, the theory has declared that they are operationally interchangeable for the chosen tasks. If they lie in different classes, there exists a task that distinguishes them.

There is therefore no remaining ambiguity at the level of exact deterministic task behavior. One can certainly ask a different question—for example, "how much energy does it take to distinguish them?"—but that is not a missing semantic premise. It is a new resource problem.

18.2 Resource closure

Likewise, once an architecture class and resource budget are specified, the capacity theorem has a complete logical target:

Can this architecture supply enough configurations?

A negative answer yields impossibility. A positive answer does not promise a working implementation because other architectural constraints may intervene.

The asymmetry is intentional:

lower bounds require proof of necessity;
feasibility requires construction or a separate sufficiency theorem.

18.3 The endpoint

The methodology therefore stops at a meaningful boundary:

$$\text{semantic quotient} \quad \rightarrow \quad \text{architecture theorem.}$$

No additional notion of "rigor of the rigor" is required to validate the mathematical argument at this level.

This convergence property is not an attempt to make the theory philosophically complete. It is a deliberate restriction that keeps the framework useful, testable, and resistant to infinite regress.

19 Discussion: What the Framework Actually Establishes

The framework supports several statements of increasing strength.

19.1 Level I: semantic statement

A specification induces a behavioral quotient. Its classes are exactly the states that remain distinguishable by the permitted future tasks.

19.2 Level II: exact state statement

Any exact deterministic state-based realization must distinguish different classes, and the quotient itself realizes the specification. Therefore

$$D(S)$$

is the exact minimum abstract state count.

19.3 Level III: architecture-capacity statement

If an architecture under budget $\mathbf{B}$ can supply at most $\mathrm{Cap}_{\mathcal{A}}(\mathbf{B})$ configurations, then

$$D(S) > \mathrm{Cap}_{\mathcal{A}}(\mathbf{B})$$

is an impossibility certificate.

19.4 Level IV: resource-cost statement

A claim that a resource itself must grow requires a separate theorem

$$C^{\star}_{\mathcal{A}}(S) \geq g_{\mathcal{A}}(D(S)).$$

Only then can a semantic increase be converted into a certified lower-bound increase.

19.5 Level V: engineering-optimum statement

Even a valid lower bound need not characterize the true optimum. The optimum may be strictly larger because of computation, timing, reliability, robustness, manufacturing, software, or other constraints.

These levels should never be collapsed into one sentence.

> *The framework is strongest when it says exactly what has been proved and stops exactly where the proof stops.*

20 Conclusion

The motivating engineering intuition can be formalized without adopting the false slogan that more rigorous mathematics is necessarily less useful.

For a deterministic task semantics,

$$F_S : \mathcal{X} \times \mathcal{T} \to \mathcal{Y},$$

the behavioral signature

$$\sigma_S(x) = F_S(x, \cdot)$$

induces an operational equivalence relation and a canonical quotient

$$\mathcal{X}/\!\sim_S .$$

Its cardinality

$$D(S) = |\mathcal{X}/\!\sim_S|$$

is simultaneously the number of task-distinguishable behavioral classes and, in the unrestricted deterministic state model, the exact minimum number of abstract states required for realization.

The engineering lower bound enters only afterward. If the chosen architecture offers fewer configurations than the quotient requires, exact realization is impossible:

$$D(S) > \mathrm{Cap}_{\mathcal{A}}(\mathbf{B}) \quad \Longrightarrow \quad \text{impossibility.}$$

For b binary state bits this becomes

$$\boxed{b \geq \lceil \log_2 D(S) \rceil .}$$

For refinement sequences, the semantic state count multiplies and its logarithm adds:

$$\boxed{\log_2 D_n = \log_2 D_0 + \sum_i \log_2 q_i.}$$

But this accumulation is not a theorem that total engineering cost rises monotonically. A strict cost claim requires a separately justified architecture-specific resource lower bound.

For approximate realization, the same logic survives in metric form. If

$$M_S(2\varepsilon) > K,$$

then K configurations cannot achieve worst-case behavioral error ε. Covering numbers provide the complementary constructive perspective in the unrestricted response model. The resulting theory replaces an all-or-nothing exactness slogan by a quantitative trade-off among capacity, precision, and approximation.

The central principle can therefore be stated one final time:

A mathematical refinement is not intrinsically resource-costly. It becomes certifiably resource-bearing when it creates task-relevant behavioral distinctions and a specified implementation class admits a proven resource lower bound that increases with those distinctions.

The paper makes no universal claim connecting mathematical rigor to engineering cost. It proposes something narrower and more durable: a semantics-to-resource methodology.

First define the tasks. Then quotient away distinctions that no admissible task can observe. Identify the exact semantic state requirement. Specify the architecture and all allowed information channels. Derive the appropriate capacity or resource lower bound. Only then decide whether the design needs more resources, a different architecture, fewer task-relevant distinctions, or a controlled relaxation of exactness.

The resulting research question is concrete:

For a given application, how sharply can minimum realization resources be characterized from task semantics?

That question is open enough to support substantial theory and concrete engineering work, yet sufficiently bounded that its claims can be proved, tested, and challenged without an infinite regress of ever more abstract notions of rigor.

A Notation Summary

Symbol	Meaning
$\mathcal{X}$	information-state space
$\mathcal{T}$	admissible future-task family
$\mathcal{Y}$	outcome space
$F_S(x,t)$	exact task outcome required by S
$\sigma_S(x)$	future-behavior signature of x
$\sim_S$	operational equivalence under S
Σ_S	set of distinct behavioral signatures
$D(S)$	operational distinguishability count
$\mathcal{Z}_I$	configuration space of implementation I
ϕ_I	state-to-configuration map
G_I	implementation response map
$\mathrm{Cap}_{\mathcal{A}}(\mathbf{B})$	configuration capacity of architecture $\mathcal{A}$ under budget $\mathbf{B}$
$C^{\star}_{\mathcal{A}}(S)$	minimum scalar exact-realization resource
$g_{\mathcal{A}}$	certified lower-bound function
$\mathrm{Dist}(S)$	$\log_2 D(S)$, worst-case distinguishability index
$M_S(\alpha)$	α-packing number of behavioral signatures
$N_S(\varepsilon)$	ε-covering number of behavioral signatures
$\varepsilon^{\star}$	optimum or threshold worst-case approximation error, with the subscript specifying the model when necessary

B Binary Threshold Table

For exact state-based realization, a configuration requirement D yields:

D	2	3	4	5	8	10	16	100	1000
$\lceil \log_2 D \rceil$	1	2	2	3	3	4	4	7	10

The table illustrates a key distinction: semantic distinguishability can increase continuously on a logarithmic scale even while the minimum integer number of binary bits remains unchanged between thresholds.

C Proof Audit and Logical Dependency Map

For reference, the core results depend on remarkably few premises.

C.1 Exact theory

1. F_S is deterministic.
2. The task family $\mathcal{T}$ is fixed.
3. Exact realization means equality of all required task outputs.
4. The implementation is state-based through ϕ_I.

Under those assumptions:

$$\text{equivalence} \Rightarrow \text{quotient} \Rightarrow \text{state separation} \Rightarrow \text{quotient realization}.$$

No probability distribution, physical law, thermodynamic principle, or coding theorem is required.

C.2 Capacity theory

Add:

1. an implementation class $\mathcal{A}$;
2. a resource budget $\mathbf{B}$;
3. a justified configuration-capacity function.

Then:

$$D(S) > \mathrm{Cap}_{\mathcal{A}}(\mathbf{B}) \Rightarrow \text{exact impossibility}.$$

C.3 Approximation theory

Add:

1. a metric on outcomes;
2. an induced metric on behavioral signatures;
3. a worst-case error criterion.

Then:

$$M_S(2\varepsilon) > K \Rightarrow K \text{ configurations cannot achieve error } \varepsilon.$$

In the unrestricted response model,

$$N_S(\varepsilon) \leq K \Rightarrow \text{a } K\text{-configuration } \varepsilon\text{-realization exists.}$$

This dependency map is intended to make the framework auditable. Each conclusion can be traced backward to the exact assumptions required to obtain it.

D Design Checklist

Before asserting that a mathematical refinement creates a hardware or software resource burden, verify the following:

1. **Task relevance:** What future task actually distinguishes the cases?
2. **Semantic state count:** What is $D(S)$, or what lower bound on it is available?
3. **Realization model:** What persistent information and channels are allowed?
4. **Architecture:** What configurations can the implementation actually provide?
5. **Resource law:** What independently justified theorem connects distinguishability to the resource being discussed?
6. **Approximation:** Is exact equality genuinely required, or should a behavioral metric and packing/covering analysis be used?
7. **Counterexamples:** Could compression, recomputation, external information, task restriction, or architecture substitution remove the claimed cost increase?

A resource claim that survives this checklist is substantially stronger than the original informal intuition that "more mathematical rigor costs more."

E Closing Statement

The durable claim of the Pulickal Framework is therefore not that mathematics and engineering are antagonists. It is that the interface between them can be analyzed cleanly when one distinguishes:

what must be distinguished

from

how a chosen architecture realizes those distinctions.

The first question is semantic. The second is architectural. The lower bound appears only at their interface.

That separation is the framework's main discipline, its main source of rigor, and also its mechanism of convergence.

References

[1] R.-J. Back and J. von Wright, *Refinement Calculus: A Systematic Introduction*, Springer, 1998.

[2] P. Cousot and R. Cousot, "Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints," in *Proceedings of POPL*, pp. 238–252, 1977.

[3] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed., SIAM, 2002.

[4] A. C. Antoulas, *Approximation of Large-Scale Dynamical Systems*, SIAM, 2005.

[5] J. Myhill, "Finite automata and the representation of events," Wright Air Development Command Technical Report 57–624, 1957.

[6] A. Nerode, "Linear automaton transformations," *Proceedings of the American Mathematical Society*, vol. 9, no. 4, pp. 541–544, 1958.

[7] E. F. Moore, "Gedanken-experiments on sequential machines," in C. E. Shannon and J. McCarthy (eds.), *Automata Studies*, pp. 129–153, Princeton University Press, 1956.

[8] F. W. Vaandrager and A. Midya, "A Myhill–Nerode theorem for register automata and symbolic trace languages," *Theoretical Computer Science*, vol. 912, pp. 37–55, 2022.

[9] A. N. Kolmogorov and V. M. Tikhomirov, "ε-entropy and ε-capacity of sets in function spaces," *Uspekhi Matematicheskikh Nauk*, vol. 14, no. 2(86), pp. 3–86, 1959.

[10] T. Knoth et al., "Resource-Guided Program Synthesis," arXiv:1904.07415, 2019.

[11] E. M. Clarke, O. Grumberg, and D. Peled, *Model Checking*, MIT Press, 1999.

[12] E. M. Clarke, O. Grumberg, D. Kroening, D. Peled, and H. Veith, *Model Checking*, 2nd ed., MIT Press, 2018.

[13] G. Piccinini, *Physical Computation: A Mechanistic Account*, Oxford University Press, 2015.

[14] E. Chitambar and G. Gour, "Quantum resource theories," *Reviews of Modern Physics*, vol. 91, 025001, 2019.

[15] K. Gödel, "Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I," *Monatshefte für Mathematik und Physik*, vol. 38, pp. 173–198, 1931.

[16] J. F. Whidborne and P. Chevrel, "LMI formulations for minimal sensitivity finite word length controller realizations," *IFAC Proceedings Volumes*, vol. 44, no. 1, pp. 780–785, 2011.